Project 14 – TSP Branch and Bound

Unit V – Branch and Bound

Group 12 - 24WH1A0501

DAA Macro Project 2026

Problem Statement

Traveling Salesperson Problem (TSP)

- Given n cities and travel costs between them
- Find the minimum cost Hamiltonian cycle
- Visit every city exactly once and return to start
- NP-Hard — Branch and Bound reduces search space via pruning

Input – Cost Matrix (4 Cities)

	C0	C1	C2	C3
C0	∞	10	8	9
C1	10	∞	5	7
C2	8	5	∞	6
C3	9	7	6	∞

Algorithm – Branch and Bound

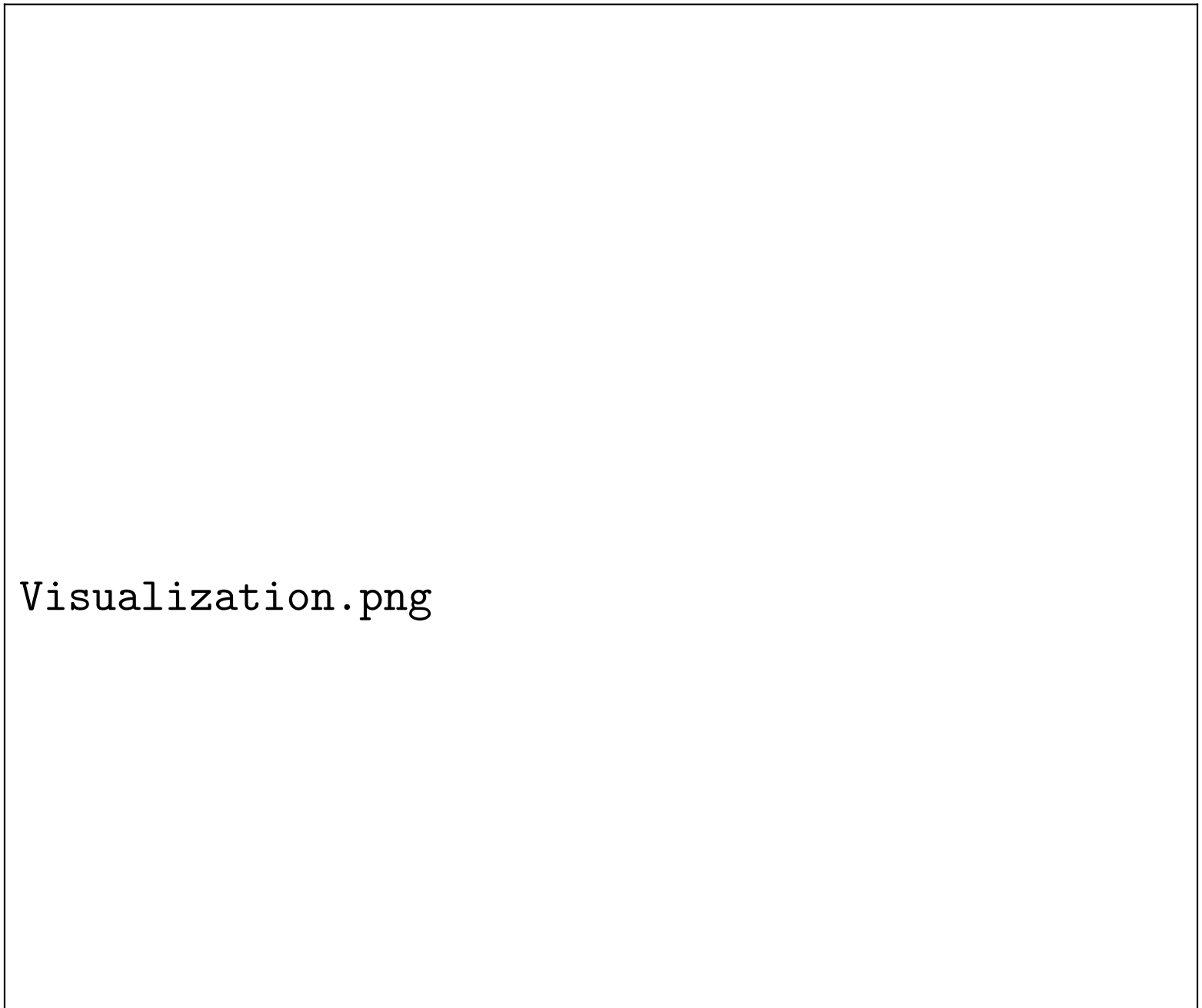
1. Reduce cost matrix (row + column reduction) $\rightarrow$ initial lower bound
2. Push root node (City 0) into a min-heap priority queue
3. Pop node with minimum bound; for each unvisited city j:
   - Build child matrix: block row[current] and col[j]
   - Compute child bound = parent bound + edge cost + reduction cost
   - **Prune** if child bound $\geq$ best known cost
   - Else push child into queue
4. When all cities visited, record tour cost if it improves best

Pseudocode

```
reduce_matrix(M):
    for each row: subtract row_min
    for each col: subtract col_min
    return M, total_reduction

tsp_bnb():
    M0, bound0 = reduce_matrix(COST)
    pq = min_heap[(bound0, city=0, path=[0], M0)]
    best = INF
    while pq not empty:
        bound, city, path, M = pop(pq)
        if bound >= best: continue          // prune
        if len(path) == n:
            best = min(best, bound + M[city][0])
        for j not in path:
            child_M = block(M, city, j)
            child_bound = bound + M[city][j] + reduce(child_M)
            if child_bound < best: push(pq, child_bound, j, path+[j])
    return best
```

Search Tree Visualization



Output

Result

- **Optimal Path:** City0 $\rightarrow$ City3 $\rightarrow$ City1 $\rightarrow$ City2 $\rightarrow$ City0
- **Minimum Cost:** 29
- **Nodes Explored:** 6 (pruning eliminated remaining branches)

Complexity

Time	$O(n^2 \cdot 2^n)$ worst case
Space	$O(n^2 \cdot 2^n)$
Pruning	Significantly reduces average case

Learning Outcomes

- Understood Branch and Bound as a systematic search strategy
- Learned matrix reduction to compute valid lower bounds
- Understood how pruning reduces the search space
- Practiced prompt-based visualization and GitHub documentation